2026-5-29 C++二叉搜索树

- 1. 二叉搜索树的概念
- 2. 二叉搜索树的性能分析
- 3. 二叉搜索树的实现
  - 3.1二叉搜索树节点创建
  - 3. 2二叉树搜索树的插入
  - 3.3 二叉搜索树的查找
- 3.4 二叉搜索树的删除
- 5. 二叉搜索树key/value使用场景
  - 5.1 key搜索场景
  - 5.2 key/value搜索场景
  - 5.3 Key/value二叉搜索树代码实现
    - 5.3.1 节点结构定义
    - 5.3.2 key/value的构造函数
    - 5.3.3 operator=的定义
    - 5.3.4 析构函数
    - 5.3.5 插入函数
    - 5.3.6 查找函数
    - 5.3.7 删除函数

1. 二叉搜索树的概念

二叉搜索树又称二叉排序树，它可以是一棵空树，或者是一棵具有以下性质的二叉树。

1. 如果该树的左子树不为空，则左子树上所有节点的值都小于等于根节点的值
2. 如果该树的右子树不为空，则右子树上所有节点的值都大于等于根节点的值
3. 其左右子树也分别为二叉搜索树
4. 它既可以支持插入相等的值也可以支持插入不相等的值，具体看使用的场景。

2. 二叉搜索树的性能分析

1. 最好的情况下，二叉搜索树为完全二叉树（或者接近完全二叉树）其高度为$\log_2(N)$
2. 最差的情况下，二叉搜索树退化为单支树（类似于单支），其高度为：N
   综合二者而言，二叉搜索树的增删改查的时间复杂度为：O(N)后面我们介绍AVL树和红黑树效率更高，更加适用于我们在内存中存储和搜索数据。

其中二分查找也可以实现$\log_2(N)$级别的查找效率，但是查找效率有两大缺陷：

1. 需要存储在支持下标随机访问的结构中，并且有序
2. 插入和删除数据的效率很低，因为二分查找依托于顺序存储结构，向该存储结构中插入亦或者是删除数据都涉及到数据的挪动。
   由此，就体现出了二叉搜索树的价值。

3. 二叉搜索树的实现

3.1二叉搜索树节点创建

1. 首先分析二叉树节点中有那些元素？我们有一个值key定义为模板类型，左右指针分别指向key的左右孩子。定义成员变量。
2. 写一个构造函数，使用初始化列表对上述节点进行初始化。
   创建二叉搜索树类：

```
template<class K>
struct BSTNode {
    //首先创建节点定义三个变量
    K _Key;
    BSTNode<K>* _right;
    BSTNode<K>* _left;
    //定义构造函数
    BSTNode(const K& Key)
        :_Key(Key)
        , _right(nullptr)
        , _left(nullptr) {

    }
};
```

3. 2二叉树搜索树的插入

为了维持二叉搜索树的特征，我们应该如何在二叉搜索树中插入元素呢？

1. 首先，我们要做的不是急着插入，而是先判断根节点是否为空，如果根节点为空，则new一个Node对象，该Key存储进去，返回`ture`.
2. 我们创建两个变量：`parent`,`cur`对该二叉搜索树进行遍历，`cur`首先指向根节点，如果根节点的值大于Key,则cur往根节点的左孩子那边移动，如果根节点的值小于key，则cur往右边移动直到cur指向为空我们就找到了该值的需要插入的位置。那我们可以直接为这个位置赋值吗？答案当然是不可以的，我们插入的主要逻辑是将该节点连接到这棵二叉搜索树

上，那么我们就需一直记录着插入位置的父亲节点。
代码如下（整个框架在这里展示一下）：

```
template<class K>
struct BSTTree {
public:
    typedef BSTNode<K> Node;//简化名称
    //首先写一个插入函数
    bool Insert(const K& Key) {
        //首先判断根节点是否为空,为空则创建节点将该值存入为根节点
        if (_root == nullptr) {
            _root = new Node(Key);
            return true;
        }
        //根节点不为空，则寻找插入位置
        Node* cur = _root;
        Node* parent = nullptr;
        while (cur) {
            if (Key < cur->_Key) {
                parent = cur;
                cur = parent->_left;
            }
            else if (Key > cur->_Key) {
                parent = cur;
                cur = parent->_right;
            }
            else {
                return false;
            }
        }
        cur= new Node(Key);
        //找到了要插入的对应位置，开始插入
        if (Key > parent->_Key) {
            parent->_right = cur;
        }
        else {
            parent->_left = cur;
        }
        return true;
    }
private:
    Node* _root = nullptr;

};
```

3.3 二叉搜索树的查找

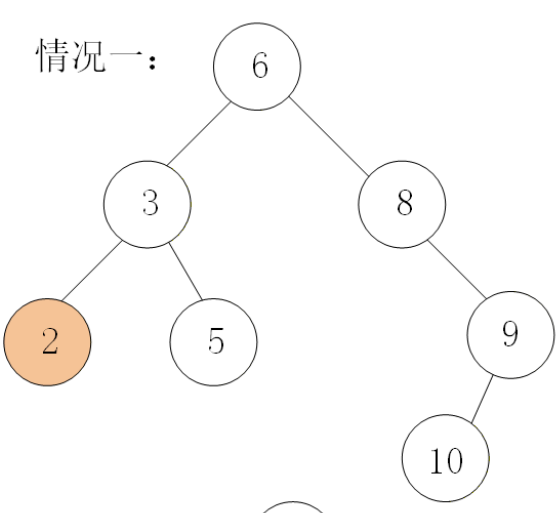
我们如何在二叉在二叉搜索树中查找元素呢?

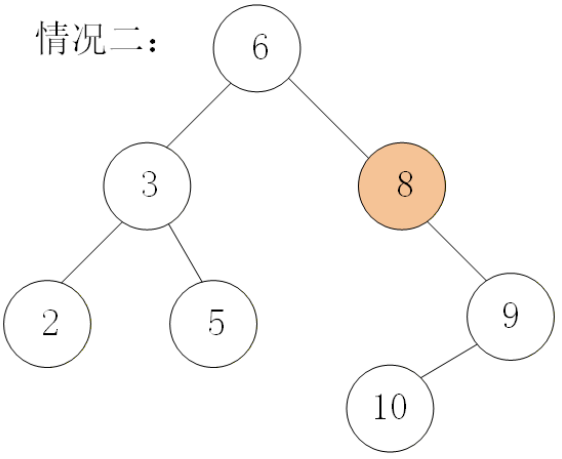
1. 首先我们需要定义一个cur变量来对该二叉搜索树进行遍历
2. 整体的的逻辑遵循从根节点开始遍历，也就是cur从根节点开始，比cur大的cur向右移动，比cur小的cur向左移动，知道cur为空截止，找到返回true没找到返回false。

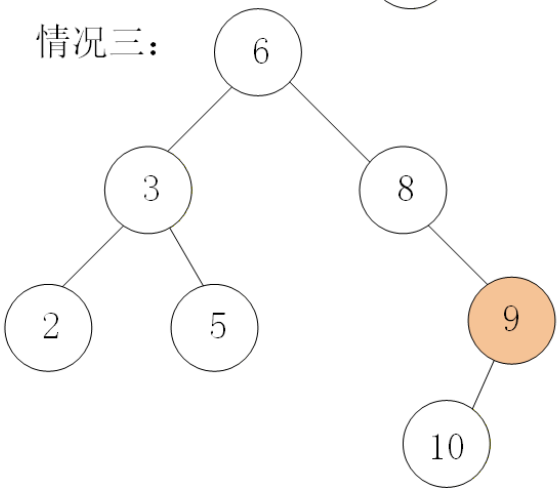
```
bool Find(const K& Key) {
    Node* cur = _root;
    while (cur) {
        if (Key > cur->_Key) {
            cur = cur->_right;
        }
        else if (Key < cur->_Key) {
            cur = cur->_left;
        }
        else {
            cout << "找到了" << endl;
            return true; }
    }
    cout << "没找到" << endl;
    return false;
}
```

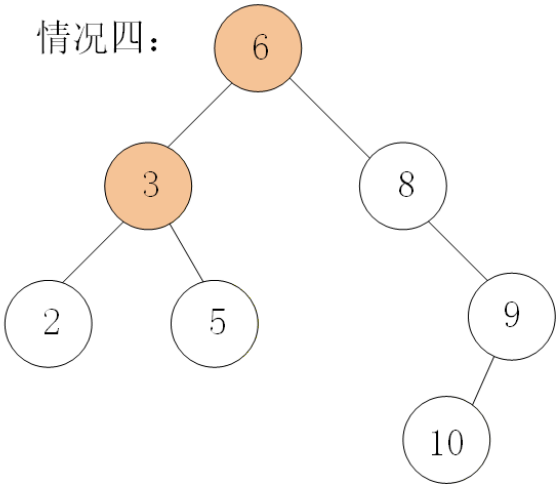
3.4 二叉搜索树的删除

关于二叉搜索树的删除逻辑情况就比较多了,我们分为以下四种情况：









首先要定义两个变量`parent`,`cur`， `cur`来遍历整个二叉平衡树
第一种情况：要删除节点的左右孩子都为空

1. 直接将`cur`所对应的节点`delate`
2. 将`parent`所指向的`cur`原本的位置置为空，防止野指针的出现

第二种情况：要删除的节点左孩子不为空

1. 首先判断`cur`是`parent`的左孩子还是右孩子，然后让`cur`的左孩子替代它自己的位置
2. 将`cur`给`delate`

第三种情况：要删除的节点的右孩子不为空

1. 首先判断`cur`是`parent`的左孩子还是右孩子，然后让`cur`的右孩子替代它自己的位置
2. 将`cur`给`delate`

第四种情况：要删除节点的左右孩子都不为空

1. 首先明确，我们的思路是找一个节点来替代要删除节点的位置，这个节点的值要求比左子树所有节点的值都大，比右子树所有节点的值都小。那就有两个备选方案：找要删除节点的左子树的最大节点（最右节点）或者找右子树最小节点，二者任意找一个替代上去都不会破坏掉二叉搜索树原有的结构。
2. 将要删除的节点的值交换或者直接覆盖掉，然后问题就转化成了问题二和问题三。
   在具体代码的实现中，实际上问题一可以合并为问题二或者问题三当中的其中一种中，可以把它想成也有两个孩子嘛，只是这两个孩子的值为空。
   具体实现情况如下：

```
template<class K>
struct BSTTree {
public:
    typedef BSTNode<K> Node;

    bool Insert(const K& Key) {
        if (_root == nullptr) {
            _root = new Node(Key);
            return true;
        }
        Node* cur = _root;
        Node* parent = nullptr;
        while (cur) {
            if (Key < cur->_Key) {
                parent = cur;
                cur = parent->_left;
            }
            else if (Key > cur->_Key) {
                parent = cur;
                cur = parent->_right;
            }
            else {
                return false;
            }
        }
        cur = new Node(Key);
        if (Key > parent->_Key) {
            parent->_right = cur;
        }
        else {
            parent->_left = cur;
        }
        return true;
    }
```

```
    bool Find(const K& Key) {
        Node* cur = _root;
        while (cur) {
            if (Key > cur->_Key) {
                cur = cur->_right;
            }
            else if (Key < cur->_Key) {
                cur = cur->_left;
            }
            else {
                cout << "找到了" << endl;
                return true;
            }
        }
        cout << "没找到" << endl;
        return false;
    }

    bool Erase(const K& Key) {
        Node* cur = _root;
        Node* parent = nullptr;

        while (cur) {
            if (Key > cur->_Key) {
                parent = cur;
                cur = cur->_right;
            }
            else if (Key < cur->_Key) {
                parent = cur;
                cur = cur->_left;
            }
            else {
                // 找到了要删除的节点 cur，parent 是其父节点
                // 情况1：左孩子为空
                if (cur->_left == nullptr) {
                    if (parent == nullptr) {
                        // 删除的是根节点
                        _root = cur->_right;
                    }
                    else {
                        if (parent->_left == cur) {
                            parent->_left = cur->_right;
                        }
                        else {
                            parent->_right = cur->_right;
                        }
```

```
                }
                delete cur;
                return true;
            }
            // 情况2：右孩子为空
            else if (cur->_right == nullptr) {
                if (parent == nullptr) {
                    _root = cur->_left;
                }
                else {
                    if (parent->_left == cur) {
                        parent->_left = cur->_left;
                    }
                    else {
                        parent->_right = cur->_left;
                    }
                }
                delete cur;
                return true;
            }
            // 情况3：左右孩子都不为空
            else {
                // 找右子树的最小节点（最左节点）
                Node* rightMinParent = cur;// 不能设为 nullptr
                //因为 rightMin 可能就是 cur->_right
                Node* rightMin = cur->_right;
                while (rightMin->_left) {
                    rightMinParent = rightMin;
                    rightMin = rightMin->_left;
                }
                // 替换值
                cur->_Key = rightMin->_Key;
                // 删除 rightMin 节点
                if (rightMinParent->_left == rightMin) {
                    rightMinParent->_left = rightMin->_right;
                }
                else {
                    rightMinParent->_right = rightMin->_right;
                }
                delete rightMin;
                return true;
            }
        }
    }
    return false; // 未找到要删除的节点
}
```

```
private:
    Node* _root = nullptr;
};
```

关于上述代码中，有两个需要特别关注的坑需要注意：

1. 情况一和情况而是将要删除节点的孩子托付给他自己的父母，但如果要删除的这个节点正好没有父母呢？那就直接让该节点的孩子节点替换根节点，最后delate该节点即可
2. 情况三中如果要删除节点的右子树的根就是最小的情况（情况四删除节点六会出现该情况）,如果这个时候将代码逻辑写成下面的这种。

```
// 错误示例：假设 rightMin 一定有左孩子
Node* rightMinParent = cur;
Node* rightMin = cur->_right;
while (rightMin->_left) {
    rightMinParent = rightMin;
    rightMin = rightMin->_left;
}
cur->_Key = rightMin->_Key;
rightMinParent->_left = rightMin->_right;
delete rightMin;
```

如果 `rightMin` 就是 `cur->_right`（即右子树的根节点就是最小节点），那么 `rightMinParent` 就是 `cur`此时执行 `rightMinParent->_left = rightMin->_right`，实际上修改的是 `cur->_left`（左孩子指针），而不是 `cur->_right`（右孩子指针)结果：**右子树的结构没有被正确修改**，而是错误地动了左子树。
判断 `rightMinParent->_left == rightMin` 来决定改左还是改右，完美覆盖了这个边界情况，这是正确的解决方式。

5. 二叉搜索树key/value使用场景

5.1 key搜索场景

只有key作为关键码，结构中只需要存储key即可，关键码即为需要搜索到的值，搜索场景只需要判断key在不在。key的搜索场景实现的二叉搜索树支持增删查，但是不支持修改，修改key会破坏搜索树原有的结构。
二叉搜索树的运用场景：

1. 小区中无人值守的车库，小区车库买了车位的车主才能开车进入小区，系统里会存储购买了车位的车主的车的车牌号，车辆在进入车库时车牌不在系统中，车辆才能进入，否则不能进入。
2. 当我们需要检查一篇英文文章单词拼写是否正确的时候，我们可以将词库中所有单词都存入二叉搜索树中，注意读取文章中的单词进行查找，查找单词是否在搜索树中。

5.2 key/value搜索场景

每一个关键码都会有一个对应的`value`， `value`可以是任意类型的对象。树的结构中（节点)除了需要存储key还要存储对应的`value`值，增/删/改/查开始以`key`为关键字走二叉搜索树的规则进行比较，这样可以快速查找到`key`对应的`value`值。`key/value`的搜索场景实现二叉搜索树的支持修改，但是不支持修改`key`,修改`key`会破化搜索树的性质，可以修改`value`。`key/value`搜索场景举例：

1. 简单的中英互译字典，树结构中存储`key`（英文）和`value`(中文)，搜索时值需要输入英文就能得到对应的中文。
2. 商场的车库，树结构中存储车牌号作为关键字`key`， `value`可以记录该车牌对应车辆的的入场时间，出场时间来计算出停车时间，从而计算车费等。
3. 统计一篇因为文章中单词的出现次数，读取一个单词`key`,查找该单词是否存在，不存在则说明该单词第一次出现将该单词存储为`key`将`value`记为1，如果单词存在则`value++`。

5.3 Key/value二叉搜索树代码实现

5.3.1 节点结构定义

1. 这个时候节点中会存储两个值，定义为模板类型k,和v
2. 定义构造函数，这个和key二叉树的方式一样，只需要记得v也需要初始化就可。

```
template<class K,class V>
struct BSTNode {
    K _Key;
    V _Value;
    BSTNode<K>* _right;
    BSTNode<K>* _left;

    BSTNode(const K& Key,const V& Value)
        :_Key(Key)
        ,_Value(Value)
        , _right(nullptr)
```

```
        , _left(nullptr) {
    }
};
```

5.3.2 key/value的构造函数

1. 构造
2. 拷贝构造--将_root复制过来就可

```
BSTTree() = default;
// 拷贝构造
BSTTree(const BSTTree<K, V>& t) {
    _root = Copy(t._root);
}
```

5.3.3 operator=的定义

类似于之前list复制重载的现代写法，交换并返回`*this`即可

```
BSTTree<K, V>& operator=(BSTTree<K, V> t) {
    swap(_root, t._root);
    return *this;
}
```

5.3.4 析构函数

1. Destroy根节点并
2. 将根节点置为空

```
~BSTTree() {
    Destroy(_root);
    _root = nullptr;
}
```

5.3.5 插入函数

和key搜索是一样的，仅仅只是new节点的时候多传递一个参数即可。

```
bool Insert(const K& Key, const V& Value) {
    if (_root == nullptr) {
        _root = new Node(Key, Value);
        return true;
    }
    Node* cur = _root;
    Node* parent = nullptr;
    while (cur) {
        if (Key < cur->_Key) {
            parent = cur;
            cur = parent->_left;
        }
        else if (Key > cur->_Key) {
            parent = cur;
            cur = parent->_right;
        }
        else {
            return false;
        }
    }
    cur = new Node(Key, Value);
    if (Key > parent->_Key) {
        parent->_right = cur;
    }
    else {
        parent->_left = cur;
    }
    return true;
}
```

5.3.6 查找函数

和key搜索是一样的

```
Node* Find(const K& Key) {
    Node* cur = _root;
    while (cur) {
        if (Key > cur->_Key) {
            cur = cur->_right;
        }
        else if (Key < cur->_Key) {
            cur = cur->_left;
        }
```

```
        else {
            cout << "找到了" << endl;
            return cur;
        }
    }
    cout << "没找到" << endl;
    return nullptr;
}
```

5.3.7 删除函数

和key搜索是一样的

```
bool Erase(const K& Key) {
    Node* cur = _root;
    Node* parent = nullptr;

    while (cur) {
        if (Key > cur->_Key) {
            parent = cur;
            cur = cur->_right;
        }
        else if (Key < cur->_Key) {
            parent = cur;
            cur = cur->_left;
        }
        else {
            // 情况1：左孩子为空
            if (cur->_left == nullptr) {
                if (parent == nullptr) {
                    _root = cur->_right;
                }
                else {
                    if (parent->_left == cur) {
                        parent->_left = cur->_right;
                    }
                    else {
                        parent->_right = cur->_right;
                    }
                }
                delete cur;
                return true;
            }
            // 情况2：右孩子为空
```

```
            else if (cur->_right == nullptr) {
                if (parent == nullptr) {
                    _root = cur->_left;
                }
                else {
                    if (parent->_left == cur) {
                        parent->_left = cur->_left;
                    }
                    else {
                        parent->_right = cur->_left;
                    }
                }
                delete cur;
                return true;
            }
            // 情况3：左右孩子都不为空
            else {
                Node* rightMinParent = cur;
                Node* rightMin = cur->_right;
                while (rightMin->_left) {
                    rightMinParent = rightMin;
                    rightMin = rightMin->_left;
                }
                // 替换值（注意也要替换 Value）
                cur->_Key = rightMin->_Key;
                cur->_Value = rightMin->_Value;
                // 删除 rightMin 节点
                if (rightMinParent->_left == rightMin) {
                    rightMinParent->_left = rightMin->_right;
                }
                else {
                    rightMinParent->_right = rightMin->_right;
                }
                delete rightMin;
                return true;
            }
        }
    }
    return false;
}
```

完整代码如下：

```
#pragma once
#include<iostream>
```

```
using namespace std;

template<class K, class V>
struct BSTNode {
    K _Key;
    V _Value;
    BSTNode<K, V>* _right;
    BSTNode<K, V>* _left;

    BSTNode(const K& Key, const V& Value)
        :_Key(Key)
        , _Value(Value)
        , _right(nullptr)
        , _left(nullptr) {
    }
};

template<class K, class V>
class BSTTree {
public:
    typedef BSTNode<K, V> Node;

    BSTTree() = default;
    // 拷贝构造
    BSTTree(const BSTTree<K, V>& t) {
        _root = Copy(t._root);
    }



    ~BSTTree() {
        Destroy(_root);
        _root = nullptr;
    }
    // 赋值重载（现代写法）
    BSTTree<K, V>& operator=(BSTTree<K, V> t) {
        swap(_root, t._root);
        return *this;
    }

    bool Insert(const K& Key, const V& Value) {
        if (_root == nullptr) {
            _root = new Node(Key, Value);
            return true;
        }
        Node* cur = _root;
        Node* parent = nullptr;
```

```
        while (cur) {
            if (Key < cur->_Key) {
                parent = cur;
                cur = parent->_left;
            }
            else if (Key > cur->_Key) {
                parent = cur;
                cur = parent->_right;
            }
            else {
                return false;
            }
        }
        cur = new Node(Key, Value);
        if (Key > parent->_Key) {
            parent->_right = cur;
        }
        else {
            parent->_left = cur;
        }
        return true;
    }

    bool Find(const K& Key) {
        Node* cur = _root;
        while (cur) {
            if (Key > cur->_Key) {
                cur = cur->_right;
            }
            else if (Key < cur->_Key) {
                cur = cur->_left;
            }
            else {
                cout << "找到了" << endl;
                return true;
            }
        }
        cout << "没找到" << endl;
        return false;
    }

    bool Erase(const K& Key) {
        Node* cur = _root;
        Node* parent = nullptr;

        while (cur) {
```

```
            if (Key > cur->_Key) {
                parent = cur;
                cur = cur->_right;
            }
            else if (Key < cur->_Key) {
                parent = cur;
                cur = cur->_left;
            }
            else {
                // 情况1：左孩子为空
                if (cur->_left == nullptr) {
                    if (parent == nullptr) {
                        _root = cur->_right;
                    }
                    else {
                        if (parent->_left == cur) {
                            parent->_left = cur->_right;
                        }
                        else {
                            parent->_right = cur->_right;
                        }
                    }
                    delete cur;
                    return true;
                }
                // 情况2：右孩子为空
                else if (cur->_right == nullptr) {
                    if (parent == nullptr) {
                        _root = cur->_left;
                    }
                    else {
                        if (parent->_left == cur) {
                            parent->_left = cur->_left;
                        }
                        else {
                            parent->_right = cur->_left;
                        }
                    }
                    delete cur;
                    return true;
                }
                // 情况3：左右孩子都不为空
                else {
                    Node* rightMinParent = cur;
                    Node* rightMin = cur->_right;
                    while (rightMin->_left) {
```

```
                        rightMinParent = rightMin;
                        rightMin = rightMin->_left;
                    }
                    // 替换值（注意也要替换 Value）
                    cur->_Key = rightMin->_Key;
                    cur->_Value = rightMin->_Value;
                    // 删除 rightMin 节点
                    if (rightMinParent->_left == rightMin) {
                        rightMinParent->_left = rightMin->_right;
                    }
                    else {
                        rightMinParent->_right = rightMin->_right;
                    }
                    delete rightMin;
                    return true;
                }
            }
        }
        return false;
    }

    void InOrder() {
        _InOrder(_root);
        cout << endl;
    }

private:
    Node* _root = nullptr;

    void _InOrder(Node* root) {
        if (root == nullptr) {
            return;
        }
        _InOrder(root->_left);
        cout << root->_Key << ":" << root->_Value << endl;
        _InOrder(root->_right);
    }

    void Destroy(Node* root) {
        if (root == nullptr) {
            return;
        }
        Destroy(root->_left);
        Destroy(root->_right);
        delete root;
    }
```

```
    Node* Copy(Node* root) {
        if (root == nullptr)
            return nullptr;
        Node* newRoot = new Node(root->_Key, root->_Value);
        newRoot->_left = Copy(root->_left);
        newRoot->_right = Copy(root->_right);
        return newRoot;
    }
};
```